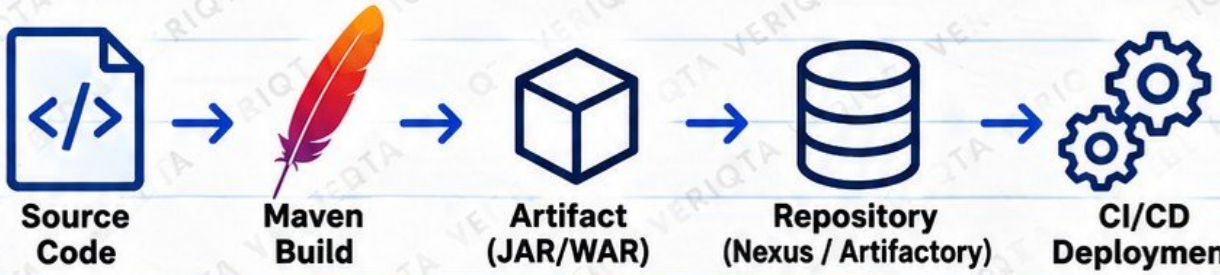
VERIQTA

LEARN
BUILD
PRACTICE
GROW

Maven™ HANDBOOK

A COMPLETE JUNIOR ENGINEER GUIDE TO BUILD, MANAGE AND DELIVER JAVA APPLICATIONS WITH MAVEN



- STEP BY STEP EXPLANATIONS
- REAL COMMANDS AND EXAMPLES
- TROUBLESHOOTING GUIDES
- REAL WORLD DEVOPS CONTEXT

FROM BASICS TO PRODUCTION

- ✅ BEGINNER FRIENDLY
- ✅ PRACTICAL EXAMPLES
- ✅ DEVOPS FOCUSED

VERIQTA
ENGINEER TODAY A BETTER TOMORROW

MAVEN: WHERE IT FITS IN DEVOPS

UNDERSTAND MAVEN, ITS ROLE, AND HOW IT FITS INTO THE DEVOPS WORKFLOW

1 WHAT APACHE MAVEN IS

- Apache Maven is an open source build automation tool for Java projects.
- It helps you manage project structure, dependencies, build lifecycle and create a deployable application.
- Maven uses a file called pom.xml (Project Object Model) to define how your project is built.
- It follows a standard structure that makes Java projects consistent and easy to automate.

2 THE PROBLEM MAVEN SOLVES

- Managing Java dependencies manually is complex and time consuming.
- Different developers may use different versions, causing build failures.
- It is easy to make mistakes when compiling, testing and packaging by hand.
- Maven solves this by automating the entire build process, managing dependencies, and using a standard project structure.

3 JAVA SOURCE CODE VS A BUILT APPLICATION

Java Source Code

- Human readable files (.java)
- Contains your application logic
- Needs to be compiled before it can run

Maven Build

Compile + Package

Built Application

- Compiled bytecode (.class) packaged as a JAR or WAR
- Can be executed or deployed
- Ready to run in a server, container or cloud environment

4 BUILD AUTOMATION

- Build automation means using tools to perform repeatable tasks.
- Maven automates compile, test, package, dependency download and more.
- You run simple commands (for example mvn package) and Maven handles the rest.
- This saves time, reduces human error and ensures consistent builds across teams.

5 MAVEN'S ROLE IN DEVOPS

- Maven builds the application and produces an artifact (JAR or WAR).
- It is used in CI/CD pipelines to automate builds.
- It ensures consistent builds across environments (development, test, production).
- It integrates with tools like Jenkins, Git, Artifactory and container platforms.
- Maven helps DevOps teams deliver reliable, repeatable and traceable releases.

6 DEVOPS FLOW WITH MAVEN

Developer	Git	Maven	Artifact	CI/CD	Deployment
Writes code	Stores code in a repository	Builds the application	JAR / WAR file	Tests, packages and deploys	Runs in server, container or cloud

7 MAVEN VERSUS GIT, JENKINS AND ARTIFACTORY

Tool	What It Is	Main Purpose	How It Relates to Maven
Maven	Build automation tool	Compiles, tests and packages Java applications	Produces the artifact (JAR/WAR) used by other tools
Git	Version control system	Stores and tracks source code	Maven builds the code from the repository
Jenkins	CI/CD automation server	Automates build, test and deployment pipelines	Runs Maven commands (for example mvn clean package)
Artifactory	Artifact repository manager	Stores and manages build artifacts and dependencies	Maven can publish and download artifacts from Artifactory

KEY TAKEAWAY Maven turns your Java source code into a deployable artifact and plays a key role in the DevOps toolchain by enabling consistent, automated and repeatable builds.

INSTALLING MAVEN AND UNDERSTANDING YOUR ENVIRONMENT

GET JAVA AND MAVEN SET UP CORRECTLY TO START BUILDING PROJECTS

1 JAVA DEVELOPMENT KIT (JDK) REQUIREMENT

- Maven is a build tool for Java applications.
- You must have the Java Development Kit (JDK) installed before using Maven.
- JDK provides the Java compiler (javac) and runtime (java).
- Maven works with JDK 8, 11, 17 or later (most projects use 11 or 17).
- Check your project or organization requirements.

JDK must be installed first

2 INSTALLING JAVA AND MAVEN

- On macOS (using Homebrew)

```
brew install openjdk
brew install maven
```

- On Ubuntu / Debian

```
sudo apt update
sudo apt install openjdk-17-jdk
sudo apt install maven
```

- On Windows
  - Download JDK and Maven from official websites
  - Install and set environment variables
  - Or use a package manager like Chocolatey

Install JDK first, then Maven

3 CHECK VERSIONS

- Check Java version

```
java -version
```

- Check Maven version

```
mvn -version
```

You should see the installed version details for both Java and Maven.

4 ENVIRONMENT VARIABLES

Variable	What It Does
JAVA_HOME	Points to your JDK installation directory
MAVEN_HOME	Points to your Maven installation directory
PATH	Includes paths to the java and mvn executables so you can run them from any directory

Example (macOS / Linux)

```
export JAVA_HOME=/usr/lib/jvm/java-17-openjdk
export MAVEN_HOME=/usr/local/opt/maven
export PATH=$JAVA_HOME/bin:$MAVEN_HOME/bin:$PATH
```

Add these to your ~/.bashrc or ~/.zshrc

5 COMMON INSTALLATION FAILURES

- **java: command not found**
  JDK is not installed or JAVA_HOME is not set.
- **mvn: command not found**
  Maven is not installed or MAVEN_HOME is not set.
- **Wrong Java version**
  Your project may require a specific JDK version.
- **Environment variable not updated**
  You installed Java or Maven but did not add them to PATH.
- **Permission denied**
  Use sudo on Linux or check installation directory permissions.
- **Multiple Java versions**
  Use the correct JAVA_HOME or a version manager.

Read the error message carefully. It usually tells you what is missing.

6 PRACTICE: VERIFY A WORKING ENVIRONMENT

- [x] 1. Run java -version
  Confirm the JDK version is installed.
- [x] 2. Run mvn -version
  Confirm Maven is installed.
- [x] 3. Check JAVA_HOME

```
echo $JAVA_HOME
```

- [x] 4. Check MAVEN_HOME

```
echo $MAVEN_HOME
```

- [x] 5. Check PATH contains java and mvn

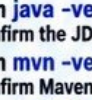
```
echo $PATH
```

- [x] 6. Create a test Maven project

```
mvn archetype:generate -DgroupId=com.veriqta \
-DartifactId=test-app -DarchetypeArtifactId=maven-archetype-quickstart \
-DinteractiveMode=false
```

- [x] 7. Navigate to the project and build it

```
cd test-app
mvn clean package
```



If the build succeeds, your environment is ready!

KEY TAKEAWAY

Install the JDK first, set your environment variables correctly, verify the versions, and run a test Maven project.

YOUR FIRST MAVEN PROJECT

CREATE A SIMPLE MAVEN PROJECT AND UNDERSTAND ITS STRUCTURE

1 WHAT MAKES A PROJECT A MAVEN PROJECT

- A project is a Maven project when it has a pom.xml file in the root directory.
- The pom.xml (Project Object Model) contains project information, dependencies, build settings and plugins.
- Maven uses this file to understand how to build your project.
- Without a pom.xml, Maven will not recognize the directory as a Maven project.

pom.xml

pom.xml = Maven project configuration file.

2 STANDARD DIRECTORY STRUCTURE

This is the standard Maven directory structure used in most Java projects.

- Your application source code
- Configuration files, properties, etc.
- Unit and integration tests
- Test configuration files
- Build output (JAR, classes, etc.)

3 CREATING A SIMPLE PROJECT

1. Open your terminal and navigate to a folder

```
cd ~/projects
```

2. Create a new Maven project using the archetype plugin

```
mvn archetype:generate \
 -DgroupId=com.veriqta \
 -DartifactId=my-app \
 -DarchetypeArtifactId=maven-archetype-quickstart \
 -DinteractiveMode=false
```

This command creates a complete Maven project with the standard structure.

3. Navigate to the new project

```
cd my-app
```

4. View the project files

```
ls -R
```

4 KEY DIRECTORIES EXPLAINED

Directory	Purpose
src/main/java	Contains your application source code (Java files).
src/main/resources	Contains configuration files, properties, and other resources.
src/test/java	Contains test code (unit and integration tests).
src/test/resources	Contains test configuration files.
target/	Contains compiled classes, test results and the packaged artifact (JAR/WAR).

5 WHY STANDARD STRUCTURE MATTERS

- Maven expects a standard structure, so you do not need to configure every path manually.
- It makes projects consistent and easy for teams to work with.
- Build tools like Jenkins, GitHub Actions and IDEs (IntelliJ, Eclipse, VS Code) understand this structure automatically.
- Your project can move between environments (developer, CI, production) without changes.
- It reduces errors and makes automation simple.

Standard structure = easier automation, faster delivery and fewer problems.

6 PRACTICE: VERIFY YOUR PROJECT

- [x] 1. Confirm the pom.xml file exists.
  `ls pom.xml`
- [x] 2. Check the project structure.
  `tree -L 2`
- [x] 3. Build the project.
  `mvn clean package`
- [x] 4. Check the target directory.
  `ls target`
- [x] 5. You should see a JAR file like:
  `my-app-1.0-SNAPSHOT.jar`

If you can build the project and see a JAR file in the target/ directory, your Maven project is working correctly.

KEY TAKEAWAY

A Maven project is defined by a pom.xml file and follows a standard directory structure. This consistency allows Maven and DevOps tools to automate your build, test and deployment process.

UNDERSTANDING pom.xml

THE BLUEPRINT OF YOUR MAVEN PROJECT

1 PROJECT OBJECT MODEL (POM)

- POM stands for **Project Object Model.**
- It is an XML file named **pom.xml** located in the root of a Maven project.
- It contains all the information Maven needs to build, test, package and manage your project.
- The POM defines your project's metadata, dependencies, build settings and plugins.

pom.xml

POM = a configuration file for your Maven project.

2 WHY MAVEN NEEDS A POM

- Tells Maven what your project is.
- Defines project metadata and dependencies.
- Configures build settings, plugins and profiles.
- Ensures consistent builds across different environments and team members.
- Allows automation tools (Jenkins, GitHub Actions, etc.) to build your project without manual steps.

Without a **pom.xml**, Maven does not know how to build your project.

3 KEY ELEMENTS IN pom.xml

Element	Description
modelVersion	Specifies the POM schema version (usually 4.0.0).
groupId	Identifies the organization or group (e.g. com.veriqta).
artifactId	Name of your project (e.g. my-app).
version	Version of your project (e.g. 1.0.0).
packaging	Type of build output (jar, war, etc).
Project metadata	Additional information like name, description, URL, developers, etc.

4 EXAMPLE pom.xml (BASIC)

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
         https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <groupId>com.veriqta</groupId>
  <artifactId>my-app</artifactId>
  <version>1.0.0</version>
  <packaging>jar</packaging>

  <name>My App</name>
  <description>A simple Maven project</descri
  <url>https://www.veriqta.com</url>
</project>
```

This is a minimal pom.xml. Real projects usually include dependencies, plugins and more configuration.

5 PROJECT METADATA (COMMON ELEMENTS)

Element	Purpose
<name>	Human readable project name.
<description>	Short description of the project.
<url>	Project website or repository URL.
<developers>	Information about project developers.
<properties>	Define reusable values (e.g. Java version).
<dependencies>	List of project dependencies.
<build>	Build plugins and configuration.

6 READING A POM WITHOUT FEAR

1. Start with the top section (modelVersion, groupId, artifactId, version, packaging).
2. Look at the project metadata (name, description).
3. Check the dependencies to see what libraries your project uses.
4. Look at the build section to understand plugins.
5. You do not need to memorize everything. Read it section by section and focus on what you need for your current task.

Read with a purpose. It gets easier with practice.

KEY TAKEAWAY

The pom.xml is the heart of a Maven project. It tells Maven what to build, how to build it, and what the project depends on. Once you understand the main elements, reading a POM becomes simple.

MAVEN COORDINATES AND ARTIFACT IDENTITY

HOW MAVEN IDENTIFIES, PACKAGES AND MANAGES YOUR ARTIFACTS

1 WHAT AN ARTIFACT IS

- An artifact is the output produced by a Maven build, usually a JAR or WAR file.
- It contains your compiled code, resources and metadata, ready to run or deploy.
- Artifacts are stored in repositories (Maven Central, Nexus, Artifactory, etc.) so they can be reused by other projects.

2 MAVEN COORDINATES

Every Maven artifact is uniquely identified by:

`groupId:artifactId:version`

groupId	artifactId	version
Organization or group (e.g. com.veriqta)	Name of the project (e.g. payment-api)	Version of the artifact (e.g. 1.0.0)

3 JAR AND WAR ARTIFACTS

- JAR (Java Archive)
- Contains compiled Java classes, resources and metadata.
- Used for standalone applications or libraries.
- Common in microservices and Spring Boot applications.

- WAR (Web Application Archive)
- Contains compiled code, resources and web files (for example, JSP, HTML).
- Deployed to a servlet container like Tomcat or Jetty.
- Used for traditional web applications.

4 WHY COORDINATES MUST BE UNIQUE

- Maven uses the coordinates to uniquely identify an artifact in a repository.
- If two different projects use the same coordinates, Maven will treat them as the same artifact, which can cause conflicts.
- Unique coordinates prevent naming collisions and make dependency management reliable across teams and organizations.

Think of coordinates as the address of an artifact in the Maven ecosystem.

5 EXAMPLE

A real example of Maven coordinates:

`com.veriqta:payment-api:1.0.0`

com.veriqta	payment-api	1.0.0
groupId (organization)	artifactId (project name)	version (release version)

6 HOW REPOSITORIES USE COORDINATES

- Repositories store artifacts using the coordinates in a directory structure.
- Maven looks at the groupId, artifactId and version to find and download the correct artifact.
- The groupId is stored as a folder path (dots → slashes).

Example repository path:

```
com/veriqta/payment-api/1.0.0/
payment-api-1.0.0.jar
```

7 ARTIFACT NAMING IN CI/CD

- In CI/CD pipelines, the artifact is usually named using the artifactId and version.
- Example: `payment-api-1.0.0.jar`
- Consistent naming helps with automation, deployment and traceability.
- Many teams also include the build number or commit hash, for example: `payment-api-1.0.0-123.jar`

CI/CD TIP

Use meaningful version numbers and consistent naming so your artifacts are easy to identify and manage.

KEY TAKEAWAY

Maven coordinates uniquely identify your artifact. They allow teams and tools to store, share and use your build output reliably across the DevOps lifecycle.

UNDERSTANDING THE MAVEN BUILD LIFECYCLE

HOW MAVEN TURNS YOUR SOURCE CODE INTO A DEPLOYABLE APPLICATION

1 LIFECYCLE vs PHASE vs GOAL

- **Lifecycle** is a group of phases that achieve a specific outcome (e.g. default lifecycle).
- **Phase** is a step in the lifecycle (e.g. compile, test, package).
- **Goal** is a specific task performed by a plugin (e.g. compiler:compile).

Lifecycle → Phase → Goal

2 DEFAULT MAVEN LIFECYCLE

- The default lifecycle is used for building and deploying your application.
- It contains a series of phases that run in order.
- Each phase performs a specific set of tasks using Maven plugins.
- You normally run one phase (like **package**), and Maven automatically runs all the earlier phases for you.

3 DEFAULT LIFECYCLE PHASES

Phase	What It Does
validate	Checks that the project is correct and all necessary information is available.
compile	Compiles the source code.
test	Runs unit tests using a framework like JUnit.
package	Packages the compiled code into a JAR or WAR file.
verify	Runs additional checks to ensure the package is valid.
install	Installs the package into your local Maven repository (`~/.m2/repository`).
deploy	Uploads the package to a remote repository (e.g. Nexus, Artifactory).

4 LIFECYCLE FLOW DIAGRAM

- validate — Check project configuration
- compile — Compile source code
- test — Run unit tests
- package — Create JAR or WAR file
- verify — Run additional checks
- install — Install to local repository
- deploy — Deploy to remote repository

Each phase depends on the previous phase. When you run a phase, Maven automatically runs all the earlier phases in the lifecycle.

5 WHY mvn package RUNS EARLIER PHASES

- Maven follows a sequential order.
- When you run **mvn package**, Maven first runs **validate → compile → test**, then runs **package**.
- This ensures the code is compiled and tested before it is packaged.
- You get a consistent and reliable build without having to run each phase manually.

6 EXAMPLE: RUNNING A PHASE

```
># Build the project (runs earlier phases too)
mvn package

># Install to local repository
mvn install

># Deploy to remote repository
mvn deploy
```

You only run the phase you need. Maven takes care of the rest.

KEY TAKEAWAY

The Maven build lifecycle provides a structured and automated way to build, test, package and deploy your application. Running a later phase automatically runs all the earlier phases.

MAVEN COMMANDS YOU WILL ACTUALLY USE

PRACTICAL MAVEN COMMANDS FOR EVERYDAY DEVELOPMENT AND CI/CD

1 COMMON MAVEN COMMANDS

Command	What It Does
mvn clean	Deletes the target/ directory (cleans previous build output).
mvn compile	Compiles the source code.
mvn test	Runs unit tests.
mvn package	Packages the compiled code into a JAR or WAR file.
mvn verify	Runs additional checks to ensure the package is valid.
mvn install	Installs the package into your local Maven repository (~/.m2/repository).
mvn clean package	Cleans the project and then packages it.
mvn clean install	Cleans the project, packages it, and installs it to your local repository.

2 WHAT HAPPENS WHEN YOU RUN A COMMAND?

`mvn clean package`

↓

clean
(delete target/)

↓

compile
(compile source code)

↓

test
(run unit tests)

↓

package
(create JAR or WAR)

Maven runs the required earlier phases automatically. So when you run `mvn package`, Maven will first run validate → compile → test, and then package.

You do not need to run each phase manually.

3 WHAT CHANGES INSIDE target/

After running Maven commands, the target/ directory contains the build output.

- target/
  - classes/ — Compiled .class files
  - test-classes/ — Compiled test classes
  - generated-sources/ — Generated source code
  - generated-test-sources/ — Generated test sources
  - surefire-reports/ — Test results (if tests run)
  - your-app-1.0.0.jar — Packaged JAR file
  - your-app-1.0.0.war — Packaged WAR file (if applicable)

The exact files and folders may vary depending on your project and plugins.

4 CHOOSING THE CORRECT COMMAND

Goal	Recommended Command
Just clean old build files	`mvn clean`
Compile the code	`mvn compile`
Run tests	`nvn test`
Build the application	`mvn package`
Run checks (tests + more)	`mvn verify`
Install to local repository	`mvn install`
Full build (clean + package)	`mvn clean package`
Full build (clean + install)	`mvn clean install`

5 KEY TAKEAWAYS

- Maven commands trigger specific phases in the build lifecycle.
- You do not need to memorize commands, understand what you want to achieve and choose the right command.
- `mvn package` runs earlier phases automatically.
- The `target/` directory contains your build output.
- Use meaningful commands in your local environment and CI/CD pipelines.

Understand the purpose, not just the command.

DEPENDENCIES:
HOW MAVEN GETS EXTERNAL CODE

MANAGE, DOWNLOAD AND USE EXTERNAL LIBRARIES IN YOUR PROJECT

1 WHAT A DEPENDENCY IS

- A dependency is an external library or module that your project needs to function (e.g. logging, testing, web frameworks).
- Maven automatically downloads dependencies from repositories and makes them available in your project.
- Dependencies save time, reduce re-invention and allow you to use well-tested code.

You focus on building your application, Maven handles the external code.

2 <dependencies> AND <dependency>

Dependencies are defined inside the <dependencies> section in your pom.xml file.

```
<dependencies>
  <dependency>
    <groupId>org.springframework</groupId>
    <artifactId>spring-web</artifactId>
    <version>6.1.0</version>
  </dependency>
</dependencies>
```

Each dependency is defined using groupId, artifactId and version.

3 DEPENDENCY COORDINATES

Each dependency is uniquely identified by a coordinate:

`groupId:artifactId:version`

groupId	Identifies the organization or group (e.g. org.springframework).
artifactId	Identifies the library name (e.g. spring-web).
version	Specifies the version of the library (e.g. 6.1.0).

Example:
`org.springframework:spring-web:6.1.0`

4 DIRECT vs TRANSITIVE DEPENDENCIES

Direct Dependency

- A dependency you explicitly add in your pom.xml.
- Example: spring-web

Transitive Dependency

- Dependencies that are automatically downloaded because your direct dependency needs them.
- You do not declare them, Maven resolves them for you.
- Example: spring-web may bring in spring-core, spring-beans, etc.

5 DEPENDENCY TREE

- The dependency tree shows all direct and transitive dependencies used in your project.
- Run the command:

```
mvn dependency:tree
```

- This helps you:
  - See the full list of dependencies
  - Identify version conflicts
  - Understand what is being downloaded
  - Troubleshoot dependency issues

```
$ mvn dependency:tree
[INFO] com.veriqta:payment-api:jar:1.0.0
+- org.springframework:spring-web:jar:6.1.0
|  +- org.springframework:spring-core:jar:6.1.0
|  +- org.springframework:spring-beans:jar:6.1.0
+- ch.qos.logback:logback-classic:jar:1.4.14
```

6 WHY DEPENDENCY MANAGEMENT MATTERS TO DEVOPS

- Ensures consistent and reproducible builds across all environments (development, CI, production).
- Helps avoid version conflicts and security vulnerabilities.
- Reduces build failures caused by incompatible dependencies.
- Allows teams to control and lock dependency versions using <dependencyManagement> and BOM (Bill of Materials).
- Critical for security scanning, compliance and stable deployments.

- Stable Builds
- Better Security
- Team Consistency
- Easier Troubleshooting
- Reliable Deployments

7 KEY TAKEAWAY

Maven dependencies give you access to powerful external libraries. Understand how they work, use the dependency tree to troubleshoot, and manage your dependencies properly for secure and reliable DevOps pipelines.

DEPENDENCY SCOPES AND VERSION MANAGEMENT

USE THE RIGHT DEPENDENCIES, MANAGE VERSIONS, BUILD STABLE APPLICATIONS

1 MAVEN DEPENDENCY SCOPES

The scope defines when and where a dependency is available in your project.

Scope	Description	Common Examples
compile	Available in all stages (compile, test, runtime). This is the default scope (if not specified).	Spring Core Apache Commons Lombok (compile time)
runtime	Not needed for compilation but required at runtime.	JDBC drivers Database clients
test	Only available for testing. Not included in the final build artifact.	JUnit TestNG Mockito
provided	Needed for compilation but provided by the runtime environment (not packaged in your artifact).	Servlet API JSP API Java EE (in application server)
import	Used inside `<dependencyManagement>` to import a BOM (Bill of Materials).	Spring Boot BOM Cloud dependencies Platform BOMs

2 VERSION PROPERTIES

- Use properties to manage versions in a single place.
- Makes it easy to update versions across multiple dependencies.
- Defined inside the `<properties>` section.

```
<properties>
  <java.version>17</java.version>
  <spring.version>6.1.0</spring.version>
  <junit.version>5.10.0</junit.version>
</properties>
```

Update the version in one place, and all dependencies that use it will be updated automatically.

3 <dependencyManagement>

- Used to define and manage dependency versions.
- Does not add dependencies by itself.
- Allows you to specify versions once and use them across multiple modules.

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>org.springframework</groupId>
      <artifactId>spring-framework</artifactId>
      <version>6.1.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

Commonly used to import a BOM (Bill of Materials) like the Spring Boot BOM.

4 BOM, BILL OF MATERIALS

- A BOM is a special POM that lists a set of compatible versions for a group of related dependencies.
- Helps ensure all dependencies work well together.
- Common examples: Spring Boot BOM, AWS SDK BOM.

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-dependencies</artifactId>
  <version>3.2.0</version>
  <type>pom</type>
  <scope>import</scope>
</dependency>
```

Using a BOM reduces version conflicts and keeps your dependencies aligned.

5 VERSION CONFLICTS

- Happen when different dependencies require different versions of the same library.
- Can cause unexpected behavior, runtime errors or build failures.
- Use `mvn dependency:tree` to identify conflicts.

```
$ mvn dependency:tree
[INFO] --- maven-dependency-plugin:3.6.0:tree ---
[INFO] com.veriqta:payment-api:1.0.0
[INFO] +- org.springframework:spring-core:6.1.0
[INFO]   \- org.example:some-lib:1.0.0
[INFO]     \- org.springframework:spring-core:5.3.0 (conflict)
[WARNING] Used version 6.1.0
```

Conflicts can introduce subtle bugs. Always check the dependency tree when you see strange behavior.

6 WHY BLINDLY CHANGING VERSIONS IS RISKY

- Newer versions can introduce breaking changes.
- Your code, tests or other dependencies may not be compatible.
- It can cause runtime failures, security issues or unexpected behavior.
- Always check release notes and test thoroughly before upgrading.

A higher version number is not always better. Stability, compatibility and testing are more important than just using the latest version.

7 KEY TAKEAWAYS

- Use the correct scope for each dependency.
- Manage versions using properties or a BOM.
- Check for version conflicts regularly.
- Do not change versions blindly. Test before upgrading.
- Good dependency management leads to stable and secure applications.

MANAGE VERSIONS BUILD WITH CONFIDENCE

MAVEN REPOSITORIES: WHERE DEPENDENCIES COME FROM

FIND, DOWNLOAD AND CACHE DEPENDENCIES FOR YOUR PROJECT

1 LOCAL REPOSITORY

- Stored on your local machine.
- Default location: `~/.m2/repository`
- Contains downloaded dependencies for your projects.
- Maven first checks the local repository before going to remote repositories.

The local repository speeds up builds by caching dependencies so they are not downloaded again.

2 MAVEN CENTRAL

- The default public repository used by Maven.
- Hosted at: `https://repo1.maven.org/maven2`
- Contains millions of open source libraries and frameworks.
- Most common dependencies (e.g. Spring, Junit, Apache) are available here.

Maven Central is the standard public repository for the Maven ecosystem.

3 REMOTE REPOSITORIES

- Other repositories besides Maven Central.
- Can be public or private (e.g. company repositories like Nexus or Artifactory).
- Defined in your `pom.xml` or `settings.xml`.
- Used when you need internal or custom libraries not available in Maven Central.

Examples: Company Nexus, JFrog Artifactory, AWS CodeArtifact, GitHub Packages.

4 REPOSITORY LOOKUP FLOW

1. Check local repository (~/.m2/repository)
2. If not found, check remote repositories (e.g. Maven Central)
3. Download and cache in local repository
4. Use the dependency from local repository

Maven always checks your local repository first. If the dependency is not found, it goes to remote repositories, downloads it, and caches it locally for future use.

5 ~/.m2/repository

- Default location for your local Maven repository.
- Structure follows: `groupId/artifactId/version/`
- Each dependency is stored in its own folder.
- You can explore it to see what Maven has downloaded.

```
~/.m2/repository/
└─ org/
   └─ springframework/
      └─ spring-core/
         └─ 6.1.0/
            ├─ spring-core-6.1.0.jar
            └─ spring-core-6.1.0.pom
```

This folder contains all downloaded dependencies for your projects.

6 DOWNLOADING AND CACHING DEPENDENCIES

- When a dependency is needed, Maven downloads it from a remote repository (e.g. Maven Central).
- The dependency is stored in `~/.m2/repository` for future use.
- Subsequent builds use the cached version, even when offline.
- This reduces build time and network usage.

Maven automatically manages downloading and caching. You do not need to do this manually.

7 WHAT HAPPENS WHEN A DEPENDENCY CANNOT BE FOUND

- Maven will try all configured repositories (local, Maven Central, and any remote repositories).
- If the dependency is not found, the build fails with an error like:

```
[ERROR] Could not resolve dependencies for project
com.veriqta:payment-api:jar:1.0.0
[ERROR] Could not find artifact com.example:my-lib:jar:1.0.0
in central (https://repo1.maven.org/maven2)
```

This can happen if the dependency does not exist, the version is incorrect, or the repository is not configured properly.

8 WHY DELETING .m2 SHOULD NOT BE THE AUTOMATIC FIX

- Deleting `~/.m2` removes all cached dependencies, which means Maven has to download everything again.
- It can be useful in some cases (e.g. corrupted downloads), but it is not a real solution to dependency problems.
- If a dependency cannot be found, the issue is usually a wrong version, missing repository, or network/repository access problem.

Fix the root cause instead of always deleting .m2. It saves time and avoids repeated downloads.

PLUGINS, GOALS, AND HOW MAVEN PERFORMS WORK

UNDERSTAND HOW MAVEN PLUGINS EXECUTE GOALS DURING THE BUILD LIFECYCLE

1 DEPENDENCIES VERSUS PLUGINS

Dependencies

- External libraries your application needs to run.
- Added inside <dependencies> in pom.xml.
- Become part of your application classpath.
- Example: Spring, JUnit, Log4j.

Plugins

- Tools that perform build tasks during the build lifecycle.
- Added inside <build><plugins> in pom.xml.
- Do not become part of your application.
- Example: Compiler, Surefire, JAR.

2 WHAT ARE MAVEN PLUGINS?

- A Maven plugin is a collection of goals (tasks).
- Plugins perform specific work during the build (compile code, run tests, create JAR, etc).
- Each plugin can have multiple goals.
- You can configure plugins in the pom.xml file.
- Maven automatically runs the right plugin goals when you use lifecycle phases (for example mvn package).
- You can also run a specific plugin goal manually using the format mvn plugin:goal.

3 PLUGIN GOALS

- A goal is a specific task inside a plugin.
- Plugins can have many goals.
- Examples:
  - maven-compiler-plugin → compile
  - maven-surefire-plugin → test
  - maven-jar-plugin → jar
- You can run goals directly using mvn plugin:goal.

4 PLUGIN EXECUTION

- Plugins run goals during the build lifecycle (for example compile, test, package).
- You can configure when a goal runs using <executions> in pom.xml.
- You can also run a plugin goal manually with mvn plugin:goal.
- Maven automatically runs many plugin goals for you when you use a lifecycle phase.

5 COMMON MAVEN PLUGINS

Maven Compiler Plugin

- Compiles Java source code to bytecode.
- Goal: compile
- Configures Java version (source and target).

Example: mvn compiler:compile

Maven Surefire Plugin

- Runs unit tests.
- Goal: test
- Works with JUnit and other testing frameworks.
- Generates test reports in target/surefire-reports.

Example: mvn surefire:test

Maven JAR Plugin

- Packages compiled code into a JAR file.
- Goal: jar
- Creates the application artifact in the target/ directory.

Example: mvn jar:jar

6 mvn plugin:goal

- You can run any plugin goal directly from the command line.
- Frormat:

```
mvn plugin:goal
```

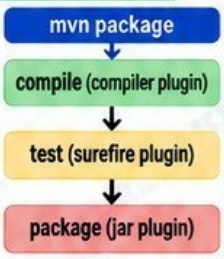
- Example:

```
mvn compiler:compile
```

- Useful for running a specific task without the entire build lifecycle.

7 HOW LIFECYCLE PHASES TRIGGER PLUGIN GOALS

- Each lifecycle phase is bound to specific plugin goals.
- When you run a phase like mvn package, Maven automatically runs the required plugin goals in the correct order.
- You do not need to run each goal manually.
- Example: mvn package typically runs:
  - compiler:compile → compiles code
  - surefire:test → runs tests
  - jar:jar → creates the JAR file



8 KEY TAKEAWAYS

- [x] Dependencies are libraries, plugins are tools.
- [x] Plugins contain goals.
- [x] Plugins run goals during the build lifecycle.
- [x] Common plugins: compiler, surefire, jar.
- [x] You can run goals manually with mvn plugin:goal.
- [x] Lifecycle phases automatically trigger the right plugin goals.
- [x] Plugins make Maven powerful and flexible.

JUNIOR ENGINEER TIP You rarely need to run individual plugin goals. Learn the lifecycle phases first, and run specific goals only when you need to debug or perform a special task.

TESTING AND BUILD QUALITY

WRITE TESTS. BUILD WITH CONFIDENCE. DELIVER BETTER SOFTWARE.

1 UNIT TESTS IN MAVEN

- Unit tests check small parts of your code (for example, a single method).
- They verify that your code works as expected.
- Common frameworks: JUnit, TestNG.
- Tests are usually located in:

```
src/test/java
src/test/resources
```

JUnit Tests
Write tests to find problems early.

2 RUNNING TESTS WITH MAVEN

- Use the mvn test command to run all tests in your project.

```
mvn test
```

- Maven compiles the code, runs the tests (using Surefire) and shows the results.
- If tests pass, you will see **BUILD SUCCESS**.
- If tests fail, you will see **BUILD FAILURE**.

3 MAVEN SUREFIRE

- Maven Surefire is the plugin responsible for running unit tests.
- It automatically detects and runs tests (for example, files ending with *Test.java).
- Generates test reports in the target/surefire-reports directory.
- Configured in the pom.xml (but works out of the box for most projects).

4 TEST REPORTS

- After running tests, Maven creates reports in:

```
target/surefire-reports/
```

- Reports include:
  - Number of tests run
  - Number of failures and errors
  - Detailed failure messages
  - Time taken
- You can open the reports in a browser to see which tests failed and why.

5 BUILD FAILURE CAUSED BY TESTS

- If any test fails, Maven stops the build and shows **BUILD FAILURE**.
- The output shows which test failed and the reason (for example, an assertion error).
- This prevents broken code from being packaged and deployed.

Example output:

```
[ERROR] Tests run: 5, Failures: 1, Errors: 0
[ERROR] BUILD FAILURE
```

6 SKIPPING TESTS (AND THE RISK)

- You can skip tests using:

```
mvn package -DskipTests
```

- This compiles the code but does NOT run the tests.
- Useful for quick builds, but it can be dangerous because:
  - Bugs can go unnoticed.
  - Broken code may be deployed.
  - You lose the safety net that tests provide.

ONLY SKIP TESTS WHEN YOU HAVE A VALID REASON.

7 WHY CI SHOULD FAIL WHEN TESTS FAIL

- CI (Continuous Integration) should normally fail if important tests fail because:
  - It prevents broken code from being merged.
  - It keeps the codebase stable and reliable.
  - It forces developers to fix issues early.
  - It saves time, cost and risk in later stages (deployment and production).
- A failing build is a signal that something needs attention, not something to ignore.

8 PRACTICE THIS

1. Create a simple Java project with a test class (for example, using JUnit).
2. Run mvn test and check the output.
3. Open the test report in target/surefire-reports/.
4. Intentionally make a test fail and observe the BUILD FAILURE message.
5. Try mvn package -DskipTests and confirm that tests are skipped.

JUNIOR ENGINEER TIP Good tests give you confidence to deploy. Do not skip tests in CI unless you fully understand the risk.

PACKAGING JAR AND WAR APPLICATIONS

COMPILE YOUR CODE. CREATE A DEPLOYABLE ARTIFACT. DELIVER TO PRODUCTION.

1 WHAT PACKAGING MEANS

- Packaging tells Maven what type of output to create from your source code.
- It is defined in the pom.xml using the <packaging> tag (for example, jar or war).
- The output is a deployable artifact that contains your compiled code, resources and dependencies (in some cases).
- Packaging makes it easy to run, deploy and share your application.

Source Code
+
Maven Build
=
Deployable Artifact

2 JAR (JAVA ARCHIVE)

- A JAR file packages compiled Java code and resources.
- Commonly used for standalone applications (for example, Spring Boot).
- Contains your classes, resources and sometimes all dependencies (when using an executable or fat JAR).
- File extension: .jar

pom.xml example:
<packaging>jar</packaging>

3 WAR (WEB APPLICATION ARCHIVE)

- A WAR file packages your web application (for example, a Java web app).
- Designed to run on a servlet container such as Apache Tomcat, Jetty or WildFly.
- Does not usually include the servlet container (it is provided by the server).
- File extension: .war

pom.xml example:
<packaging>war</packaging>

4 EXECUTABLE JAR CONCEPTS

- An executable JAR includes everything needed to run the application.
- It contains your application code, dependencies and a main class.
- Commonly used with Spring Boot.
- You can run it directly using java -jar.
- File is usually larger because it contains dependencies.

Example:
```
java -jar myapp-1.0.0.jar
```

Java Executable JAR

5 TARGET/ DIRECTORY

- Maven creates the build output in the target/ directory.
- This directory contains the compiled classes, tests results and the final artifact (JAR or WAR).
- The target/ directory is recreated when you run mvn clean.
- You should not commit the target/ directory to Git (add it to .gitignore).

Example structure:

```
target/
  |— classes/
  |— test-classes/
  |— myapp-1.0.0.jar  (or .war)
```

6 ARTIFACT NAMING

- Maven names the artifact using artifactId, version and package type.
- Format:

```
artifactId-version.packaging
```

- Example:
  - myapp-1.0.0.jar
  - myapp-1.0.0.war
- You can customize the your pom.xml if needed.

Each build produces a unique and versioned artifact.

7 BUILD OUTPUT

- When you run mvn package, Maven compiles, runs tests and creates the artifact.
- Example command:

```
mvn clean package
```

- After a successful build, you will see:

BUILD SUCCESS

8 INSPECTING GENERATED ARTIFACTS

- You can inspect the contents of a JAR using:

```
jar -tf target/myapp-1.0.0.jar
```

- You can inspect the contents of a WAR using:

```
unzip -l target/myapp-1.0.0.war
```

- This helps you confirm that the expected files are inside.

Always verify your artifact before deploying.

9 WHY CI/CD DEPLOYS ARTIFACTS

- Artifacts are compiled, tested and versioned outputs, which are consistent and reliable.
- Deploying artifacts ensures the same build is used across all environments.
- Source code is not ready to run and requires a build step.
- Using artifacts reduces errors and makes deployments faster and more repeatable.

JUNIOR ENGINEER TIP Always deploy the artifact, not the source code. The artifact is tested, versioned and ready for production.

MAVEN PROFILES
AND ENVIRONMENT-SPECIFIC BUILDS

USE MAVEN PROFILES TO MANAGE DIFFERENT ENVIRONMENTS CLEANLY AND SAFELY

1 WHAT PROFILES ARE

- Profiles allow you to customize your build for different environments (for example, dev, test, staging, prod).
- Each profile can have its own configuration, properties, dependencies and plugins.
- They help you maintain one project with multiple environment-specific builds.

Same code. Different configurations. That is the power of Maven profiles.

dev
test
staging
prod

2 <PROFILES> IN POM.XML

- Profiles are defined inside the `<profiles>` section in your pom.xml.

```
<profiles>
  <profile>
    <id>dev</id>
    <properties>
      <env.name>dev</env.name>
    </properties>
  </profile>
</profiles>
```

Each profile has a unique `<id>` and can define properties, dependencies and plugins.

3 ACTIVATING A PROFILE

- You can activate a profile using the `-P` flag.

```
mvn package -Pdev
```

- You can activate multiple profiles (comma separated).

```
mvn package -Pdev,aws
```

- You can also set a profile to activate automatically (for example, based on an environment variable, JDK version or operating system).

4 BUILD-TIME CONFIGURATION

- Profiles are often used to configure:
  - Environment-specific properties
  - Different dependencies
  - Different plugins
  - Resource files
  - Build settings

pom.xml

Profiles change how the project is built. They do not change your source code.

5 DEVELOPMENT VS PRODUCTION DIFFERENCES

Development Profile (dev)

- Uses local or test resources
- May include debug settings
- May use mock services
- Faster build (for example, skip heavy steps)

Production Profile (prod)

- Uses production resources
- Optimized and secure settings
- May include additional plugins (for example, code signing)
- No debug settings
- Strict dependency versions

6 WHY SECRETS SHOULD NOT BE HARDCODED IN POM FILES

- Never put passwords, API keys or sensitive information directly in pom.xml.
- POM files are stored in Git, which means secrets can be exposed.
- Use environment variables, Maven settings.xml, CI/CD secret management or a secrets manager (for example, AWS Secrets Manager).

Hardcoded secrets can lead to serious security breaches.

7 AVOIDING ENVIRONMENT-SPECIFIC BUILD SURPRISES

- Keep your profiles simple and well documented.
- Use clear naming (dev, test, staging, prod).
- Avoid too many profile-specific customizations.
- Test your builds in a CI/CD pipeline to ensure they work.
- Use the same Java and Maven versions across environments.
- Verify the active profile using the command:

```
mvn help:active-profiles
```

Always verify which profile is active before running a build.

8 KEY TAKEAWAYS

- ✅ Profiles help you build for different environments.
- ✅ Activate a profile with `-P`.
- ✅ Use profiles for environment-specific configuration.
- ✅ Do not hardcode secrets in pom.xml.
- ✅ Keep profiles simple and well documented.
- ✅ Test your profile builds in CI/CD.
- ✅ Always verify the active profile.

JUNIOR ENGINEER TIP Use Maven profiles to manage environment differences, not to hide secrets or create complex builds.

SETTINGS.XML, AUTHENTICATION, AND SECURE CONFIGURATION

MANAGE REPOSITORIES, CREDENTIALS AND ENVIRONMENTS SECURELY

1 USER SETTINGS VS PROJECT POM

User Settings (settings.xml)

- Located in your home directory (~/.m2/settings.xml)
- Contains user-specific configuration (repositories, credentials, mirrors, proxies).
- Not committed to Git.
- Different for each developer and CI environment.

Use settings.xml for sensitive information like credentials.

Project POM (pom.xml)

- Located in the project root (pom.xml).
- Contains project-specific configuration (dependencies, plugins, profiles).
- Usually committed to Git.
- Shared with the team.

Do not put passwords or tokens in pom.xml.

2 ~/.m2/settings.xml

- The settings.xml file customizes how Maven works for you.
- It can contain servers, mirrors, repositories and other configurations.
- Location (Linux/Mac): ~/.m2/settings.xml
- Location (Windows): C:\Users\<username>\.m2\settings.xml

Example settings.xml (simplified)

```
<settings>
  <servers>
    <server>
      <id>artifactory</id>
      <username>${env.ARTIFACTORY_USER}</username>
      <password>${env.ARTIFACTORY_TOKEN}</password>
    </server>
</settings>
```

3 SERVERS (CREDENTIALS)

- The <servers> section stores credentials for repositories like Nexus or Artifactory.
- Each server has a unique <id> that matches the repository configuration.

```
<servers>
  <server>
    <id>artifactory</id>
    <username>${env.ARTIFACTORY_USER}</username>
    <password>${env.ARTIFACTORY_TOKEN}</password>
  </server>
</servers>
```

Use environment variables instead of hardcoding usernames and passwords.

4 MIRRORS

- Mirrors allow you to redirect requests to another repository (for example, use Artifactory as a mirror of Maven Central).

```
<mirrors>
  <mirror>
    <id>company-mirror</id>
    <url>https://artifactory.
    company.com/maven</url>
  <<mirrorOf>central</mirrorOf>
  </mirror>
```

Mirrors help improve performance and control which repositories are used.

5 REPOSITORIES

- You can define repositories in settings.xml (or in pom.xml).
- Used to resolve dependencies from internal or external sources.

```
<repositories>
  <repository>
    <id>company-repo</id>
    <url>https://artifactory.
    company.com/maven-release</url>
  </repository>
</repositoriies>
```

Repositories tell Maven where to download dependencies from.

6 ENVIRONMENT VARIABLES

- Use environment variables to store sensitive values like usernames, tokens or API keys.
- Reference them in settings.xml using ${env.VARIABLE}.

Example:

```
<username>${env.ARTIFACTORY_USER}</username>
<password>${env.ARTIFACTORY_TOKEN}</password>
```

This keeps secrets out of your files and your Git repository.

7 CI SECRETS

- In CI/CD tools (Jenkins, GitHub Actions, GitLab CI, etc.), store credentials as secrets or environment variables.
- Maven will use these values during the build.
- Never hardcode secrets in your code, pom.xml or settings.xml.

Use the CI tool's secret management feature (for example, GitHub Secrets).

8 SECURITY REMINDER

- Never commit passwords, tokens or sensitive information to Git.
- Use environment variables or CI secrets.
- Review your pom.xml and settings.xml before sharing.
- If a secret is accidentally committed, rotate it immediately.

Hardcoded credentials can lead to security breaches, unauthorized access and data loss.

JUNIOR ENGINEER TIP Keep your builds secure. Use settings.xml for credentials, environment variables for sensitive values, and never commit secrets to Git.

MAVEN WITH NEXUS AND JFROG ARTIFACTORY

MANAGE DEPENDENCIES. PUBLISH ARTIFACTS. DELIVER TO PRODUCTION.

1 WHY ORGANIZATIONS USE ARTIFACT REPOSITORIES

- Store and manage internal and external artifacts in a central repository.
- Provide security, control, reliability and high availability.
- Cache public dependencies (for example, Maven Central).
- Enable consistent and reproducible builds.
- Integrate with CI/CD pipelines.
- Manage access, permissions and auditing.

Artifact repositories are essential for enterprise DevOps and supply chain security.

Sonatype Nexus

JFrog Artifactory

2 PROXYING MAVEN CENTRAL

- A proxy repository caches artifacts from Maven Central.
- Requests for public dependencies are first checked in the proxy.
- If not found, the repository downloads the artifact from Maven Central and caches it.
- This reduces build time, saves bandwidth and provides stability.

Maven Central

Maven Project → Proxy Repository → Maven Central (internet)

3 TYPES OF REPOSITORIES

Hosted (Local)	• You deploy (publish) your internal artifacts. • Stored inside Nexus or Artifactory. • Used for your own builds and releases.
Remote	• Connects to an external repository (for example, Maven Central). • Caches artifacts from the remote source. • Used when you do not want direct internet access from your build servers.
Virtual (Group)	• Combines multiple repositories (hosted and remote). • Provides a single URL for your Maven project. • Commonly used in enterprise environments.

4 RESOLVING DEPENDENCIES

- Maven checks the repositories defined in your settings.xml or pom.xml.
- It looks in the local repository first (~/.m2/repository).
- If not found, it resolves from the configured remote or virtual repository (for example, Artifactory or Nexus).
- This ensures dependencies are downloaded from approved sources.

Maven Project → Local Repository (~/.m2) → Nexus / Artifactory (Virtual) → Maven Central (if needed)

5 PUBLISHING INTERNAL ARTIFACTS

- Use mvn deploy to publish artifacts to a hosted repository.
- Configure the repository and credentials in settings.xml.
- Artifacts can be JAR, WAR, or other packages.

```
mvn deploy
```

Publish your artifacts for reuse by other teams.

6 MAVEN → ARTIFACTORY/NEXUS → APPLICATION DELIVERY

Maven Build → JFrog Artifactory (or Nexus) → Artifact (JAR/WAR) → Deploy to Application (Servers, Containers, Kubernetes, etc.)

Maven builds the artifact. Artifactory or Nexus stores it. The artifact is then deployed to your target environment.

7 REPOSITORY AUTHENTICATION FAILURES

- Incorrect username or password.
- Missing or incorrect credentials in settings.xml.
- Using the wrong repository URL.
- Insufficient permissions (no deploy access).
- Expired tokens or API keys.
- Network or proxy issues.
- Repository not reachable.
- SSL/TLS certificate problems.

Common error messages:

- 401 Unauthorized
- 403 Forbidden
- Could not transfer artifact
- Authentication failed

8 KEY TAKEAWAYS

- [x] Use artifact repositories to control and secure dependencies.
- [x] Proxy Maven Central to improve build reliability.
- [x] Understand hosted, remote and virtual repositories.
- [x] Configure credentials securely in settings.xml.
- [x] Use mvn deploy to publish internal artifacts.
- [x] Integrate Maven with Nexus or Artifactory in CI/CD.
- [x] Troubleshoot authentication failures effectively.
- [x] Keep your repositories organized and access controlled.
- [x] Artifact repositories are critical for secure and scalable delivery.

JUNIOR ENGINEER TIP Always use a virtual repository for your builds. It gives you control, security and a single URL, while still allowing access to your internal and external artifacts.

MAVEN INSIDE CI/CD PIPELINES

AUTOMATE. BUILD. TEST. PUBLISH. DEPLOY. DELIVER WITH CONFIDENCE.

1 GIT PUSH TRIGGERS PIPELINE

- When code is pushed to the repository (for example, GitHub), it triggers the CI/CD pipeline.
- The pipeline automatically starts and runs the defined steps.
- This ensures every change is built, tested and validated.

git push → Triggers CI/CD Pipeline

2 CI/CD PIPELINE STEPS

1. Checkout (get the code)
2. Configure JDK
3. Dependency resolution
4. Compile
5. Test
6. Package
7. Scan (security/quality)
8. Publish artifact
9. Deploy

3 EXAMPLE PIPELINE FLOW

GitHub (Code Push)
↓
Jenkins / GitHub Actions (Execute Pipeline)
↓
Maven (Build, Test, Package)
↓
Artifactory / Nexus (Store Artifact)
↓
Deployment (Servers / Containers / Kubernetes)

4 KEY STEPS EXPLAINED

Checkout	Runs a git clone to get your code.
Configure JDK	Sets up the correct Java version.
Dependency resolution	Downloads dependencies from Maven Central or your artifact repository.
Compile	Compiles the Java source code.
Test	Runs unit tests (Maven Surefire).
Package	Creates the artifact (JAR or WAR) in target/.
Scan	Runs security and quality scans (for example, SonarQube, dependency scan).
Publish artifact	Uploads the artifact to Artifactory or Nexus.
Deploy	Deploys to the target environment (for example, server, container or Kubernetes).

5 MAVEN COMMANDS IN CI/CD

- mvn clean install
- mvn test
- mvn package
- mvn deploy

In CI/CD, we usually run mvn clean package (or mvn clean install) and then publish the generated artifact from the target/ directory.

6 JENKINS vs GITHUB ACTIONS

Jenkins

- Self-hosted or managed
- Highly customizable
- Works with any Git provider
- Uses Jenkinsfile
- Common in enterprise environments

GitHub Actions

- Cloud-based (fully managed)
- Integrated with GitHub
- Uses YAML workflow files
- Easy to set up
- Popular for modern DevOps workflows

Both can run the same Maven build steps.

7 WHY BUILDS SHOULD BE REPRODUCIBLE

- The same source code should always produce the same artifact.
- Ensures consistency across environments (dev, test, prod).
- Reduces deployment issues.
- Helps with debugging and rollback.
- Builds are versioned and auditable.
- Supports compliance and security requirements.

Reproducible builds build trust.

8 KEY TAKEAWAYS

- ✓ A git push can trigger the entire CI/CD pipeline.
- ✓ Maven handles build, test and package.
- ✓ Artifacts are stored in Artifactory or Nexus.
- ✓ Pipelines automate and standardize delivery.
- ✓ Always use the correct JDK version.
- ✓ Scan for security and quality issues.
- ✓ Keep builds reproducible.
- ✓ A well-configured pipeline reduces risk and speeds up delivery.

JUNIOR ENGINEER TIP Automate everything, test every change, and keep your pipeline configurations in version control.

MAVEN, DOCKER, AND CONTAINER BUILDS

BUILD THE APPLICATION. PACKAGE THE ENVIRONMENT. DELIVER TO PRODUCTION.

1 MAVEN BUILDS THE APPLICATION

- Maven compiles your source code, resolves dependencies, runs tests and packages the application (for example, a JAR file).
- The output is a deployable artifact in the `target/` directory.
- Maven focuses on the application build, not the runtime environment.

Maven™

```
mvn clean package
# Output: target/myapp-1.0.0.jar
```

2 DOCKER PACKAGES THE RUNTIME ENVIRONMENT

- Docker creates a container image that includes the application and its runtime environment (JDK, OS, libraries, etc).
- It ensures the application runs the same way in every environment (dev, test, prod).
- Docker does not build the code; it packages the built artifact.

docker

3 JAR → CONTAINER IMAGE

- Maven produces a JAR file.
- Docker copies the JAR into an image and configures how to run it.


```
FROM eclipse-temurin:17-jre
COPY target/myapp-1.0.0.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

4 MULTI-STAGE BUILD CONCEPT

- Multi-stage builds use one stage to build the application and a second, smaller stage to run it.
- This reduces the final image size and improves security.

```
# Stage 1: Build with Maven
FROM maven:3.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY . .
RUN mvn clean package -DskipTests

# Stage 2: Run the application
FROM eclipse-temurin:17-jre
WORKDIR /app
COPY --from=build /app/target/myapp-1.0.0.jar app.jar
ENTRYPOINT ["java", "-jar", "app.jar"]
```

5 DEPENDENCY CACHING

- Maven downloads dependencies from repositories (for example, Maven Central or Artifactory).
- Docker can cache these dependencies to make builds faster.
- Copy `pom.xml` first to leverage Docker layer caching.

```
COPY pom.xml .
RUN mvn dependency:go-offline
COPY src/ ./src
RUN mvn clean package -DskipTests
```

6 MAVEN IN CI VS MAVEN INSIDE DOCKER BUILDS

Maven in CI (outside Docker)

- Runs on the CI server (for example, Jenkins or GitHub Actions).
- Produces the JAR file.
- Then Docker builds the image using the JAR.

Maven inside Docker builds

- Maven runs inside a Docker build (using a Maven base image).
- Simplifies the CI setup.
- Keeps the build environment consistent.
- Useful for multi-stage builds.

7 IMAGE TAGGING

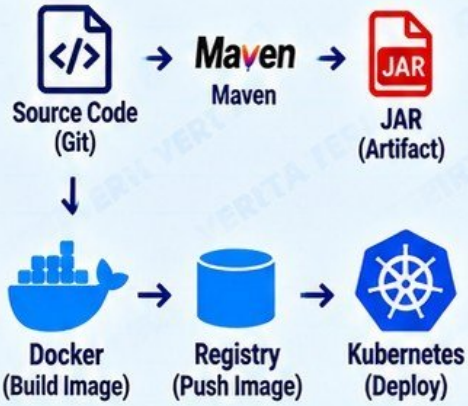
- Use meaningful tags to identify versions (for example, latest, 1.0.0, commit SHA).
- Tags help with version control, rollback and traceability.

```
docker build -t myapp:1.0.0 .

docker tag myapp :1.0.0 \\
  myapp:latest

docker push myapp:1.0.0
docker push myapp:latest
```

8 ARTIFACT VS CONTAINER IMAGE

Artifact (JAR)	Container Image
• Output of Maven build (for example, .jar or .war).	• Output of Docker build.
• Contains the application code and dependencies.	• Contains the application and runtime environment (for example, JDK + OS).
• Stored in artifact repositories (Nexus or Artifactory).	• Stored in container registries (Docker Hub, ECR, GCR, etc).
• Language and build tool specific.	• Platform independent.
• Not runnable by itself (needs a runtime).	• Runnable as a container (includes everything needed).

9 END-TO-END FLOW



TROUBLESHOOTING MAVEN LIKE AN ENGINEER

FIND THE ROOT CAUSE. FIX IT. BUILD WITH CONFIDENCE.

1 BUILD FAILURE

- **BUILD FAILURE** means Maven encountered an error and stopped.
- Read the first meaningful error message (at the top, not the last line).
- The real cause is usually above the "BUILD FAILURE" line.

```
[ERROR] BUILD FAILURE
[ERROR] ------------------------------
[ERROR] Failed to execute goal ...
[ERROR] For more information, re-run Maven with the -e switch.
```

Always fix the FIRST error, not the last one.

2 DEPENDENCY RESOLUTION FAILURES

- Maven cannot download a required dependency.
- Common causes:
  - Wrong version
  - Artifact not found
  - Repository not configured
  - Network or proxy issues
  - Private repository authentication

```
[ERROR] Could not find artifact com.example:myapp:jar:1.0.0 in central (https://repo.maven.apache.org/maven2)
```

Check the version, repository configuration and network access.

3 COULD NOT FIND ARTIFACT

- Maven cannot locate the specified artifact in any configured repository.
- Check:
  - groupId, artifactId and version
  - Repository URL
  - If the artifact actually exists
  - If you need a snapshot or release version

```
[ERROR] Could not find artifact org.example:demo:jar:1.0.0
```

Verify coordinates and repository settings (settings.xml).

4 PLUGIN FAILURES

- Maven plugins perform build tasks (compile, test, package, etc).
- Plugin failures can be caused by:
  - Wrong plugin version
  - Plugin configuration errors
  - Incompatible Java version

```
[ERROR] Failed to execute goal org.apache.maven.plugins:maven-compiler-plugin:3.10.1:compile (default-compile) on project myapp: Compilation failure
```

Check the plugin version and configuration in your pom.xml.

5 COMPILATION ERRORS

- Maven cannot compile your code (usually Java syntax or missing classes).
- Common causes:
  - Syntax errors
  - Missing dependencies
  - Wrong Java version

```
[ERROR] COMPILATION ERROR :
[ERROR] /src/main/java/App.java:[10,5] cannot find symbol
symbol:   class Example
location: class App
```

Read the error message. It tells you the file, line number and problem.

6 TEST FAILURES

- Unit tests failed (Maven Surefire).
- Check the test report for details.
- A failing test will usually stop the build.

```
[ERROR] There are test failures.
[ERROR] Please refer to /target/surefire-reports for the individual test results.
```

Open the test report to see which test failed and why.

7 JAVA/MAVEN VERSION MISMATCH

- Maven or plugins require a specific Java version.
- Check your installed versions.
- Common errors:
  - Unsupported class file major version
  - Invalid target release
  - Plugin requires a higher Java version

```
[ERROR] invalid target release: 17
```

Run java -version and mvn -version to verify.

8 AUTHENTICATION FAILURES

- Maven cannot authenticate to a private repository (Nexus, Artifactory, etc).
- Common causes:
  - Wrong username or password
  - Expired token
  - Incorrect server ID in settings.xml
  - Missing credentials in CI/CD

```
[ERROR] Failed to transfer artifact ...
401 Unauthorized
```

Check your settings.xml, credentials and token permissions.

9 NETWORK/PROXY PROBLEMS

- Maven cannot reach the repository due to network or proxy restrictions.
- Common issues:
  - No internet access
  - Incorrect proxy settings
  - Firewall blocking the connection
  - SSL/TLS certificate errors

```
[ERROR] Failed to transfer artifact ...
Connection timed out
[ERROR] Unknown host repo.maven.apache.org
```

Verify your network, proxy settings and SSL certificates.

10 USEFUL MAVEN COMMANDS

- Run Maven with debug output:

```
mvn -X
```

  - Shows detailed logs
  - Helps you see what Maven is doing
- View the dependency tree:

```
mvn dependency:tree
```

  - Shows all project dependencies
  - Helps find version conflicts
  - Useful for troubleshooting missing or incorrect dependencies

11 TROUBLESHOOTING MINDSET

1	Observe	What is the problem? Check the error message and logs.
2	Hypothesis	What could be the cause?
3	Diagnose	Gather more information (logs, config, versions).
4	Fix	Apply the correct solution.
5	Verify	Run the build again and confirm it works.

Think like an engineer. Be systematic. Find the root cause, not just a quick fix.

JUNIOR ENGINEER TIP — Most Maven errors tell you exactly what is wrong. Read carefully, stay calm, and fix one issue at a time.

PRODUCTION MAVEN PROJECT: SOURCE TO DEPLOYMENT

A COMPLETE MENTAL MODEL AROUND A REALISTIC JAVA SERVICE.

1 JAVA SOURCE

- Write your Java application (for example, a REST API using Spring Boot).
- Source code includes application logic, configuration and tests.

```
src/
├─ main/java
├─ main/resources
└─ test/java
```

2 GIT REPOSITORY

- Push your code to a Git repository (for example, GitHub).
- Version control enables tracking, collaboration and triggers CI/CD.

```
git add .
git commit -m "Initial commit"
git push
```

3 MAVEN BUILD

- Maven reads your pom.xml, downloads dependencies and prepares the build.
- It manages the entire build lifecycle.

```
mvn clean install
```

4 COMPILE + TEST

- Compiles the Java source code.
- Runs unit tests (for example, using Surefire).
- Build fails if tests fail.

```
mvn compile
mvn test
```

5 PACKAGE JAR

- Packages the compiled code into a JAR file (in the target/ directory).
- The JAR is a deployable artifact.

```
mvn package
Output: target/myapp-1.0.0.jar
```

6 SECURITY / DEPENDENCY SCAN

- Scan the JAR and dependencies for vulnerabilities (for example, SonarQube, OWASP Dependency Check, or Snyk).
- Fail the build if critical issues are found.

```
mvn sonar:sonar
(or run a security scan in CI)
```

7 PUBLISH TO ARTIFACTORY

- Upload the JAR to a repository manager (for example, JFrog Artifactory or Nexus).
- Makes the artifact available for deployment and other teams.

```
mvn deploy
Publishes to Artifactory/Nexus
```

8 BUILD DOCKER IMAGE

- Use a Dockerfile to package the application with its runtime environment.
- The JAR from the previous step is copied into the image.

```
docker build -t myapp:1.0.0 .
```

9 CONTAINER REGISTRY

- Push the Docker image to a container registry (for example, Amazon ECR, Docker Hub or Artifactory Container Registry).

```
docker push myapp:1.0.0
```

10 DEPLOY TO KUBERNETES

- Deploy the container image to Kubernetes (using manifests, Helm or a CI/CD pipeline).
- Kubernetes runs and manages your application in production.

```
kubectl apply -f deployment.yaml
(or helm install ...)
```

11 VERIFY APPLICATION

- Check that the application is running and healthy.
- Verify logs, metrics and endpoints.
- Confirm it is accessible to users.

```
kubectl get pods
kubectl logs <pod>
curl http://your-app-endpoint
```

THE BIG PICTURE

Java Source
↓
Git Repository
↓
Maven Build
↓
Compile + Test
↓
Package JAR
↓
Security Scan
↓
Publish to Artifactory
↓
Build Docker Image
↓
Container Registry
↓
Deploy to Kubernetes
↓
Verify Application

8 WHY THIS MATTERS

- ✓ Automates the entire delivery process.
- ✓ Ensures consistent and reproducible builds.
- ✓ Improves security and quality.
- ✓ Enables fast and reliable deployments.
- ✓ Supports collaboration and scalability.
- ✓ This is the standard flow used in real-world production environments.

KEY TAKEAWAYS

- Maven builds the application, not the runtime.
- Artifacts (JARs) are stored in a repository manager.
- Docker packages the application with its runtime.
- Kubernetes runs the application in production.
- Automate, test, scan and verify every step.

- Same process. Different environments (dev, test, staging, prod).
- Keep builds reproducible and versioned.
- Use automation and monitoring.
- Fix issues early in the pipeline.

"From code to customers with confidence."
— VERIQTA